Introduction to dataframes, subsetting and addressing

subtitle here

```
#install.packages("tinytext")
#tinytex::install_tinytex()
```

Factors

Factors and **ordered factors** are similar to vectors, but contain categorical data. R handles factors differently than vectors, so a factor is more than vector with a different type of data.

grades as factors

The **as.factor** function can be used to create a factor from a vector. We can demonstrate with a vector of student grades.

```
# convert character vector to factor
grades <- c("A", "B", "C", "F", "B", "A", "C", "C", "C", "B", "B", "B", "B", "A", "C", "B", "A", "F")
grades
```

```
##  [1] "A" "B" "C" "F" "B" "A" "C" "C" "C" "B" "B" "B" "B" "A" "C" "B" "A" "F"
```

```
cat("\nnow create a factor:\n")
```

```
##
## now create a factor:
```

```
grades_fact <- factor(grades)
grades_fact
```

```
##  [1] A B C F B A C C C B B B B A C B A F
## Levels: A B C F
```

```
cat("\nstr\n") # blank line
```

```
##
## str
```

```
str(grades_fact)
```

```
##  Factor w/ 4 levels "A","B","C","F": 1 2 3 4 2 1 3 3 3 2 ...
```

```r
cat("\nattributes\n") # blank line
```

```
##
## attributes
```

```r
attributes(grades_fact)
```

```
## $levels
## [1] "A" "B" "C" "F"
##
## $class
## [1] "factor"
```

The **factor** is stored in R as 1-dimensional string of levels (1,2,3,4) and those levels have assigned labels of "A", "B", "C" and "D".

A variable should be designated a factor if it is categorical and the number of categories is not too large (far fewer levels than observations.) If there are a large number of categories that are to be used descriptively, then a character vector is more appropriate.

ordered factor

For ordinal data, we want to use an **ordered factor**.

```r
grades_ord <- factor(grades, ordered = T)
grades_ord
```

```
##  [1] A B C F B A C C C B B B B A C B A F
## Levels: A < B < C < F
```

```r
cat("\n")
```

```r
str(grades_ord)
```

```
##  Ord.factor w/ 4 levels "A"<"B"<"C"<"F": 1 2 3 4 2 1 3 3 3 2 ...
```

```r
cat("\nattributes\n") # blank line
```

```
##
## attributes
```

```r
attributes(grades_ord)
```

```
## $levels
## [1] "A" "B" "C" "F"
##
## $class
## [1] "ordered" "factor"
```

The order by default is alphabetical, but we probably want A > B > C > F. Also notice that there is no "D" grade. The factor function just works with the levels given. To fix these problems we can set the levels.

```r
# set ordered=T to create an ordered factor
grades_ord2 <- factor(grades, ordered = T, levels = c("F", "D", "C", "B", "A"))
grades_ord2
```

```
##  [1] A B C F B A C C C B B B B A C B A F
## Levels: F < D < C < B < A
```

```r
cat("\n")
```

```r
str(grades_ord2)
```

```
##  Ord.factor w/ 5 levels "F"<"D"<"C"<"B"<..: 5 4 3 1 4 5 3 3 3 4 ...
```

```r
cat("\nattributes\n") # blank line
```

```
##
## attributes
```

```r
attributes(grades_ord2)
```

```
## $levels
## [1] "F" "D" "C" "B" "A"
##
## $class
## [1] "ordered" "factor"
```

Notice that we now have a "D" level even though there are no D grades in the ordered factor.

Numerical data to factors

R can also convert numerical or integer data to factors. This data is coerced into character data, then the factor function operates as illustrated previously.

```r
samp <- sample(x = c(1, 3, 4, 7, 9), size = 90, replace = T)
str(samp)
```

```
##  num [1:90] 9 7 9 3 1 4 7 7 7 1 ...
```

```r
# convert numeric data to a factor
samp_fact <- factor(samp)
cat("\nPrint factor structure\n")
```

```
##
## Print factor structure
```

```r
str(samp_fact)
```

```
##  Factor w/ 5 levels "1","3","4","7",..: 5 4 5 2 1 3 4 4 4 1 ...
```

```r
levels(samp_fact)
```

```
## [1] "1" "3" "4" "7" "9"
```

Notice that the factor levels do NOT match the factor labels!

We can also convert samp1 into an ordered factor.

```r
samp_ord <- factor(samp, ordered = T)
str(samp_ord)
```

```
##  Ord.factor w/ 5 levels "1"<"3"<"4"<"7"<..: 5 4 5 2 1 3 4 4 4 1 ...
```

Coercion with as.factor, as.ordered

If we are okay with default settings, **as.factor** can coerce a vector into a factor.

```r
as.factor(grades)
```

```
##  [1] A B C F B A C C C B B B B A C B A F
## Levels: A B C F
```

```r
cat("\n") # extra line
```

```r
as.factor(samp)
```

```
##  [1] 9 7 9 3 1 4 7 7 7 1 1 3 3 7 3 3 3 3 1 9 4 1 1 1 7 9 4 9 1 3 1 4 9 7 3 3 4 1
## [39] 3 4 4 7 9 9 7 1 3 7 3 7 1 4 1 3 1 3 4 7 7 3 1 9 1 7 9 3 9 1 1 7 9 1 1 1 4 7
## [77] 3 1 9 1 9 3 4 9 4 9 7 1 9 9
## Levels: 1 3 4 7 9
```

Similarly, we can use **as.ordered** to coerce a vector into an ordered factor.

```r
as.ordered(grades)
```

```
##  [1] A B C F B A C C C B B B B A C B A F
## Levels: A < B < C < F
```

```r
cat("\n") # extra line
```

```r
as.ordered(samp)
```

```
##  [1] 9 7 9 3 1 4 7 7 7 1 1 3 3 7 3 3 3 3 1 9 4 1 1 1 7 9 4 9 1 3 1 4 9 7 3 3 4 1
## [39] 3 4 4 7 9 9 7 1 3 7 3 7 1 4 1 3 1 3 4 7 7 3 1 9 1 7 9 3 9 1 1 7 9 1 1 1 4 7
## [77] 3 1 9 1 9 3 4 9 4 9 7 1 9 9
## Levels: 1 < 3 < 4 < 7 < 9
```

NOTES

- It makes sense to designate data as a factor or ordered factor if you have categorical or ordinal data, respectively. But you should always check to be sure there are fewer factor levels than observations, and some R functions have problems with factors that have too many levels.

Dataframes

Data frames consist of multiple vectors of equal length bound together. Each column in a data frame is like vector in that it can only hold one type of data. But the different columns in a data frame can, and usually do, hold different kinds of data. You will do most of your analysis in R working with data frames.

data.frame function

We will create a student dataframe to illustrate how the **data.frame** function works.

```
name <- c("Leila", "Vijay", "Shira", "Behrooz")
gender <- factor(c("F", "M", "F", "M"))
age <- c(18, 23, 48, 19)
height <- c(60, 70, 65, 72)
grade <- factor(c("A", "D", "B", "C"), levels = c("F", "D", "C", "B", "A"), ordered = T)

# Now put all of these factors and vectors of same length into a data frame.
students <- data.frame(name, gender, age, height, grade)
str(students)
```

```
## 'data.frame':    4 obs. of  5 variables:
##  $ name  : chr  "Leila" "Vijay" "Shira" "Behrooz"
##  $ gender: Factor w/ 2 levels "F","M": 1 2 1 2
##  $ age   : num  18 23 48 19
##  $ height: num  60 70 65 72
##  $ grade : Ord.factor w/ 5 levels "F"<"D"<"C"<"B"<..: 5 2 4 3
```

```
students
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```
head(students)
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

column names

Designate column names with data.frame

Data frame column names are generally variable names. You can designate column names in the **data.frame** function. By default, the names of the vectors used in data.frame become column names, and function names used to create data also become names.

```r
# This time set stringsAsFactors=F
students <- data.frame(first.name = name, gender, c(18, 23, 48, 19), height, grade)
students
```

```
##   first.name gender c.18..23..48..19. height grade
## 1      Leila      F                18     60     A
## 2      Vijay      M                23     70     D
## 3      Shira      F                48     65     B
## 4    Behrooz      M                19     72     C
```

We can designate the third column name as *age* using the following approach.

```r
# This time set stringsAsFactors=F
students <- data.frame(first.name = name, gender, age = c(18, 23, 48, 19), height, grade, stringsAsFacto
students
```

```
##   first.name gender age height grade
## 1      Leila      F  18     60     A
## 2      Vijay      M  23     70     D
## 3      Shira      F  48     65     B
## 4    Behrooz      M  19     72     C
```

Designate column names with colnames()

We can extract the column names of a data frame using the **colnames** function.

```r
# extract column names using colnames() function
colnames(students)
```

```
## [1] "first.name" "gender"     "age"        "height"     "grade"
```

We can also assign column names of a data frame using the **colnames** function.

```r
# assign column names using colnames() function
colnames(students) <- c("First.name", "Gender", "Age", "Height", "Grade")
students
```

```
##   First.name Gender Age Height Grade
## 1      Leila      F  18     60     A
## 2      Vijay      M  23     70     D
## 3      Shira      F  48     65     B
## 4    Behrooz      M  19     72     C
```

Address data using column names

Column names can be used to address a particular column in a data frame. (All column names in one data frame must be unique.) You can address a column using the format: *dataframe$column.name*.

```r
students$Age
```

```
## [1] 18 23 48 19
```

Change data type for one column by name.

```r
students$Age <- as.integer(students$Age)
students
```

```
##   First.name Gender Age Height Grade
## 1      Leila      F  18     60     A
## 2      Vijay      M  23     70     D
## 3      Shira      F  48     65     B
## 4    Behrooz      M  19     72     C
```

Note that we normally wouldn't designate **Age** as an integer. Even though our Age values are all integers, we know that Age is a continuous variable.

Addressing vectors

Addressing allows for concise data reshaping, extraction and assignment in R. It takes some time to master addressing as there are multiple ways to address and these ways act somewhat differently on different kinds of objects.

extraction - indexing w/ Brackets []

Square brackets are used to address and extract specific locations within a vector.

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
cat("\nextract single position (3)\n")
```

```
##
## extract single position (3)
```

```r
vect[3]
```

```
## [1] 4
```

```r
cat("\nextract sequence of positions 2:4\n")
```

```
##
## extract sequence of positions 2:4
```

```r
vect[2:4]
```

```
## [1] -2  4  6
```

```r
cat("\nextract positions 1,3,5.  c(1,3,5)\n")
```

```
##
## extract positions 1,3,5.  c(1,3,5)
```

```r
vect[c(1, 3, 5)]
```

```
## [1] 1.0 4.0 2.2
```

```r
cat("\nextract everything except position 4  c(-4)\n")
```

```
##
## extract everything except position 4  c(-4)
```

```r
vect[c(-4)]
```

```
## [1]  1.0 -2.0  4.0  2.2 12.0
```

```r
cat("\nextract everything except positions 2,3,4  -c(2:4)\n")
```

```
##
## extract everything except positions 2,3,4  -c(2:4)
```

```r
vect[-c(2:4)]
```

```
## [1]  1.0  2.2 12.0
```

Notice that we extract **values** held at the addressed positions in the vector data object.

Repeat with a character vector.

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nsingle position (3)\n")
```

```
##
## single position (3)
```

```r
pets[3]
```

```
## [1] "dog"
```

```r
cat("\nsequence of positions 1:2\n")
```

```
##
## sequence of positions 1:2
```

```r
pets[1:2]
```

```
## [1] "hamster" "cat"
```

```r
cat("\nselected positions c(1,3)\n")
```

```
##
## selected positions c(1,3)
```

```r
pets[c(1, 3)]
```

```
## [1] "hamster" "dog"
```

```r
cat("\nselected positions to remove c(-2)\n")
```

```
##
## selected positions to remove c(-2)
```

```r
pets[c(-2)]
```

```
## [1] "hamster" "dog"
```

Notice that element names remain with subsetting.

```r
# Create a vector with my height and weight in SI units (m, kg)
vect <- c("height (m)" = 1.75, "weight(kg)" = 70)
vect
```

```
## height (m) weight(kg)
##       1.75      70.00
```

```r
cat("\nselected positions (2)\n")
```

```
##
## selected positions (2)
```

```r
vect[2]
```

```
## weight(kg)
##         70
```

```r
vect <- vect[2]
vect
```

```
## weight(kg)
##         70
```

```r
length(vect)
```

```
## [1] 1
```

assignment - indexing w/ Brackets []

Addressing locations with brackets can also be used to make assignments, to store data in particular locations within a vector.

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
cat("\nassign 0 to single position (3)\n")
```

```
##
## assign 0 to single position (3)
```

```r
vect[3] <- 0
vect
```

```
## [1]  1.0 -2.0  0.0  6.0  2.2 12.0
```

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
cat("\nassign 0.9 to sequence of positions 2:4\n")
```

```
##
## assign 0.9 to sequence of positions 2:4
```

```r
vect[2:4] <- .9
vect
```

```
## [1]  1.0  0.9  0.9  0.9  2.2 12.0
```

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
cat("\nassign -5 to selected positions c(1,3,5)\n")
```

```
##
## assign -5 to selected positions c(1,3,5)
```

```r
vect[c(1, 3, 5)] <- -5
vect
```

```
## [1] -5 -2 -5  6 -5 12
```

Repeat assignment with a character vector.

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\n ferret to single position (3)\n")
```

```
##
##  ferret to single position (3)
```

```r
pets[3] <- "ferret"
pets
```

```
## [1] "hamster" "cat"     "ferret"
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
cat("\nferret to sequence of positions 1:2\n")
```

```
##
## ferret to sequence of positions 1:2
```

```r
pets[1:2] <- "ferret"
pets
```

```
## [1] "ferret" "ferret" "dog"
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
cat("\nferret, monkey to sequence of positions 1:2\n")
```

```
##
## ferret, monkey to sequence of positions 1:2
```

```r
pets[1:2] <- c("ferret", "monkey")
pets
```

```
## [1] "ferret" "monkey" "dog"
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
cat("\ngerbil, ferret to selected positions 1,3\n")
```

```
##
## gerbil, ferret to selected positions 1,3
```

```r
pets[c(1, 3)] <- c("gerbil", "ferret")
pets
```

```
## [1] "gerbil" "cat"    "ferret"
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
cat("\ngerbil, ferret to all four positions\n")
```

```
##
## gerbil, ferret to all four positions
```

```r
pets[c(1:4)] <- c("gerbil", "ferret")
pets
```

```
## [1] "gerbil" "ferret" "gerbil" "ferret"
```

Address with logical vectors & []

You can also extract with logical vectors the same length as the vector being addressed inside [].

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
cat("\nc(T,F,T,T,F,T)\n")
```

```
##
## c(T,F,T,T,F,T)
```

```r
vect[c(T, F, T, T, F, T)]
```

```
## [1]  1  4  6 12
```

```r
cat("\nc(F,F,T,F,F,F)\n")
```

```
##
## c(F,F,T,F,F,F)
```

```r
vect[c(F, F, T, F, F, F)]
```

```
## [1] 4
```

```r
cat("\n")
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nc(T,F,T)\n")
```

```
##
## c(T,F,T)
```

```r
pets[c(T, F, T)]
```

```
## [1] "hamster" "dog"
```

```r
cat("\nc(F,F,T)\n")
```

```
##
## c(F,F,T)
```

```r
pets[c(F, F, T)]
```

```
## [1] "dog"
```

```r
cat("\nc(F,F,F)\n")
```

```
##
## c(F,F,F)
```

```r
pets[c(F, F, F)]
```

```
## character(0)
```

Taking the next step, you can construct a test for vectors values that is then used to extract a subset of data using a logical vector and square brackets.

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
test <- vect > 0 # create logical vector
cat("\nvect1 >0\n")
```

```
##
## vect1 >0
```

```r
test
```

```
## [1]  TRUE FALSE  TRUE  TRUE  TRUE  TRUE
```

```r
cat("\nselect vect1 >0\n")
```

```
##
## select vect1 >0
```

```r
vect[test]
```

```
## [1]  1.0  4.0  6.0  2.2 12.0
```

```r
cat("\nselect vect1 \n")
```

```
##
## select vect1
```

```r
vect[!test] # ! means 'not' logically, so T -> F, F -> T
```

```
## [1] -2
```

```r
test <- vect > 0 # create logical vector
cat("\nvect1 >0\n")
```

```
##
## vect1 >0
```

```r
test
```

```
## [1]  TRUE FALSE  TRUE  TRUE  TRUE  TRUE
```

```r
cat("\nlength for number of T in logical vector\n")
```

```
##
## length for number of T in logical vector
```

```r
length(test[test])
```

```
## [1] 5
```

```r
cat("\nsum for number of T in logical vector\n")
```

```
##
## sum for number of T in logical vector
```

```r
sum(test)
```

```
## [1] 5
```

other addressing issues

What happens with non-standard use of square brackets [].?

```r
cat("\n")
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nNotice what we get with []\n")
```

```
##
## Notice what we get with []
```

```r
pets[]
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nNotice what we get with [0]\n")
```

```
##
## Notice what we get with [0]
```

```r
pets[0]
```

```
## character(0)
```

NA always gives NA.

```r
cat("\n")
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nc(T,NA,F)\n")
```

```
##
## c(T,NA,F)
```

```r
pets[c(T, NA, F)]
```

```
## [1] "hamster" NA
```

```r
cat("\nc(NA,2)\n")
```

```
##
## c(NA,2)
```

```r
pets[c(NA, 2)]
```

```
## [1] NA    "cat"
```

Addressing 2D data objects

Addressing applies to matrices and data frames in much the same way as to vectors.

Addressing matrices []

As with vectors, matrices can be subsetted, addressed and assigned with square brackets. THe difference is that the square brackets contain two parts [**??**,**??**], addressing row and column dimensions.

```r
# create a larger matrix using sequenced numbers
mat <- matrix(data = seq(-10, 47, by = 3), nrow = 4, ncol = 5, byrow = T)
dim(mat) <- c(5, 4)
mat
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]   20   38   -1   17
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

```r
cat("\n3rd row, 2nd column [3,2]\n")
```

```
##
## 3rd row, 2nd column [3,2]
```

```r
mat[3, 2]
```

```
## [1] 38
```

```r
cat("\n1st to 3rd row, 2nd column [1:3,2]\n")
```

```
##
## 1st to 3rd row, 2nd column [1:3,2]
```

```r
mat[1:3, 2]
```

```
## [1]  8 23 38
```

```r
cat("\n1st to 3rd row, 2nd and 3rd column [1:3,2:3]\n")
```

```
##
## 1st to 3rd row, 2nd and 3rd column [1:3,2:3]
```

```r
mat[1:3, 2:3]
```

```
##      [,1] [,2]
## [1,]    8   26
## [2,]   23   41
## [3,]   38   -1
```

```r
cat("\n1st 3rd 5th row, 3rd column [c(1,3,5),3]\n")
```

```
##
## 1st 3rd 5th row, 3rd column [c(1,3,5),3]
```

```r
mat[c(1, 3, 5), 3]
```

```
## [1] 26 -1 29
```

As with vectors, we can also remove rows or columns with negative indices. For a given dimension, all indices must be positive or all must be negative.

```r
mat # as above
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]   20   38   -1   17
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

```r
cat("\nremove 3rd row, remove 2nd column [-3,-2]\n")
```

```
##
## remove 3rd row, remove 2nd column [-3,-2]
```

```r
mat[-3, -2]
```

```
##      [,1] [,2] [,3]
## [1,]  -10   26   44
## [2,]    5   41    2
## [3,]   35   14   32
## [4,]   -7   29   47
```

```r
cat("\nremove 1st to 3rd row, 2nd column [-(1:3),2]\n")
```

```
##
## remove 1st to 3rd row, 2nd column [-(1:3),2]
```

```r
mat[-(1:3), 2]
```

```
## [1] -4 11
```

```r
cat("\n1st to 3rd row, remove 2nd and 3rd column [1:3,-(2:3)]\n")
```

```
##
## 1st to 3rd row, remove 2nd and 3rd column [1:3,-(2:3)]
```

```r
mat[1:3, -(2:3)]
```

```
##      [,1] [,2]
## [1,]  -10   44
## [2,]    5    2
## [3,]   20   17
```

```r
cat("\n1st 3rd 5th row, remove 3rd column [c(1,3,5),-3]\n")
```

```
##
## 1st 3rd 5th row, remove 3rd column [c(1,3,5),-3]
```

```r
mat[c(1, 3, 5), -3]
```

```
##      [,1] [,2] [,3]
## [1,]  -10    8   44
## [2,]   20   38   17
## [3,]   -7   11   47
```

Notice that with matrices, entire rows or columns are removed with negative indices. We can't remove a single element in a matrix.

As with vectors, leaving an index blank means all rows or all columns are retained.

```r
mat # as above
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]   20   38   -1   17
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

```r
cat("\nremove 3rd row, keep all columns [-3,]\n")
```

```
##
## remove 3rd row, keep all columns [-3,]
```

```r
mat[-3, ]
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]   35   -4   14   32
## [4,]   -7   11   29   47
```

```r
cat("\nkeep all rows, select 2nd column [,2]\n")
```

```
##
## keep all rows, select 2nd column [,2]
```

```r
mat[, 2]
```

```
## [1]  8 23 38 -4 11
```

```r
cat("\nkeep all rows, remove 2nd and 3rd column [,-(2:3)]\n")
```

```
##
## keep all rows, remove 2nd and 3rd column [,-(2:3)]
```

```r
mat[, -(2:3)]
```

```
##      [,1] [,2]
## [1,]  -10   44
## [2,]    5    2
## [3,]   20   17
## [4,]   35   32
## [5,]   -7   47
```

We can also *assign* values to specific locations with addressing brackets [].

```r
mat # as above
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]   20   38   -1   17
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

```r
cat("\nassign 0 to 2nd row, 3rd column\n")
```

```
##
## assign 0 to 2nd row, 3rd column
```

```r
mat3 <- mat
mat3[2, 3] <- 0
mat3
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23    0    2
## [3,]   20   38   -1   17
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

```r
mat2 <- mat

cat("\nassign 99 to 2nd column\n")
```

```
##
## assign 99 to 2nd column
```

```r
mat3 <- mat2
mat3[, 2] <- 99
mat3
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10   99   26   44
## [2,]    5   99   41    2
## [3,]   20   99   -1   17
## [4,]   35   99   14   32
## [5,]   -7   99   29   47
```

```r
cat("\nassign new sequence to 3rd row\n")
```

```
##
## assign new sequence to 3rd row
```

```r
mat3 <- mat2
mat3[3, ] <- c(0:-3)
mat3
```

```
##      [,1] [,2] [,3] [,4]
## [1,]  -10    8   26   44
## [2,]    5   23   41    2
## [3,]    0   -1   -2   -3
## [4,]   35   -4   14   32
## [5,]   -7   11   29   47
```

Notice that as with vectors, we can assign a vector of values to a range of locations (e.g. sequence 0, -1, -2, -3 as shown above).

Change the c(0:-3) sequence end point to -1 **c(0:-1)** and rerun to see what happens. No try for -2 and -4.

addressing dataframes

Data frames can be subsetted similar to matrices. First recreate earlier data frame.

```r
name <- c("Leila", "Vijay", "Shira", "Behrooz")
gender <- factor(c("F", "M", "F", "M"))
age <- c(18, 23, 48, 19)
height <- c(60, 70, 65, 72)
grade <- factor(c("A", "D", "B", "C"), levels = c("F", "D", "C", "B", "A"), ordered = T)

# Now put all of these factors and vectors of same length into a data frame.
students <- data.frame(name, gender, age, height, grade, stringsAsFactors = F)
students
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

Now we can extract a subset with bracket addressing.

```r
students
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```r
cat("\n2nd row, 3rd column (Vijay's age)\n")
```

```
##
## 2nd row, 3rd column (Vijay's age)
```

```r
students[2, 3]
```

```
## [1] 23
```

```r
cat("\n1st row (Leila info)\n")
```

```
##
## 1st row (Leila info)
```

```r
students[1, ]
```

```
##    name gender age height grade
## 1 Leila      F  18     60     A
```

```r
cat("\n3rd row w/o grade (Shira)\n")
```

```
##
## 3rd row w/o grade (Shira)
```

```r
students[3, -5]
```

```
##    name gender age height
## 3 Shira      F  48     65
```

And we can assign data to a dataframe position

```r
students
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```r
cat("\n2nd row, 3rd column (change Vijay's age from 23 to 32)\n")
```

```
##
## 2nd row, 3rd column (change Vijay's age from 23 to 32)
```

```r
students2 <- students
students2[2, 3] <- 32
students2
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  32     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

subsetting matrices or data frames using logical vectors

We can also extend the way we extracted subsets of vectors with logical vectors to data frames.

```
students <- data.frame(name, gender, age, height, grade, stringsAsFactors = F)
students # original data frame
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```
cat("\n1st and 3rd row [c(T,F,T,F),]\n")
```

```
##
## 1st and 3rd row [c(T,F,T,F),]
```

```
students[c(T, F, T, F), ]
```

```
##    name gender age height grade
## 1 Leila      F  18     60     A
## 3 Shira      F  48     65     B
```

```
cat("\n1st and 5th column [,c(T,F,F,F,T)]\n")
```

```
##
## 1st and 5th column [,c(T,F,F,F,T)]
```

```
students[, c(T, F, F, F, T)]
```

```
##      name grade
## 1   Leila     A
## 2   Vijay     D
## 3   Shira     B
## 4 Behrooz     C
```

```
students[students$age < 25, ]
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 4 Behrooz      M  19     72     C
```

Note that the logical vector length has to match the dimension size. We can use the same approach to address matrices.

Address variables by name

We often address dataframe variables by name using the '$' symbol.

```r
students
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```r
cat("\nstudents$age\n")
```

```
##
## students$age
```

```r
students$age
```

```
## [1] 18 23 48 19
```

```r
cat("\nstudents$grade\n")
```

```
##
## students$grade
```

```r
students$grade
```

```
## [1] A D B C
## Levels: F < D < C < B < A
```

```r
# we can combine $ and []
cat("\nnames, 2nd and 3rd $name[2:3]\n")
```

```
##
## names, 2nd and 3rd $name[2:3]
```

```r
students$name[2:3]
```

```
## [1] "Vijay" "Shira"
```

```r
cat("\nrows 1:3 $age\n")
```

```
##
## rows 1:3 $age
```

```r
students[1:3, ]$age
```

```
## [1] 18 23 48
```

Reordering vectors, matrics and dataframes

Addressing can be used to reorder without necessarily reducing data.

order vector data

Note that square brackets can also be used to reorder data.

```
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```
cat("\nc(3,1,2,4,5,6)\n")
```

```
##
## c(3,1,2,4,5,6)
```

```
vect[c(3, 1, 2, 4, 5, 6)]
```

```
## [1]  4.0  1.0 -2.0  6.0  2.2 12.0
```

```
cat("\n")
```

```
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```
cat("\nc(3,2,1)\n")
```

```
##
## c(3,2,1)
```

```
pets[c(3, 2, 1)]
```

```
## [1] "dog"     "cat"     "hamster"
```

The **order** function extracts the alphanumeric order of values in a vector.

```
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
order(vect)
```

```
## [1] 2 1 5 3 4 6
```

```r
cat("\n")
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
order(pets)
```

```
## [1] 2 3 1
```

```r
cat("\n")
```

```r
order(pets, decreasing = T)
```

```
## [1] 1 3 2
```

You can use **order** to extract the order, then sort using addressing. There is no base 'sort' function in R, though there are sorting functions available in data wrangling packages.

```r
vect <- c(1, -2, 4, 6, 2.2, 12) # create vector data object
vect # print data object to the console
```

```
## [1]  1.0 -2.0  4.0  6.0  2.2 12.0
```

```r
cat("\nsort increasing\n")
```

```
##
## sort increasing
```

```r
order(vect)
```

```
## [1] 2 1 5 3 4 6
```

```r
vect[order(vect)]
```

```
## [1] -2.0  1.0  2.2  4.0  6.0 12.0
```

```r
cat("\nsort decreasing\n")
```

```
##
## sort decreasing
```

```r
order(vect, decreasing = T)
```

```
## [1] 6 4 3 5 1 2
```

```r
vect[order(vect, decreasing = T)]
```

```
## [1] 12.0  6.0  4.0  2.2  1.0 -2.0
```

```r
cat("\n\n")
```

```r
pets <- c("hamster", "cat", "dog") # create vector data
pets
```

```
## [1] "hamster" "cat"     "dog"
```

```r
cat("\nsort increasing\n")
```

```
##
## sort increasing
```

```r
order(pets)
```

```
## [1] 2 3 1
```

```r
pets[order(pets)]
```

```
## [1] "cat"     "dog"     "hamster"
```

```r
cat("\nsort decreasing\n")
```

```
##
## sort decreasing
```

```r
order(pets, decreasing = T)
```

```
## [1] 1 3 2
```

```r
pets[order(pets, decreasing = T)]
```

```
## [1] "hamster" "dog"     "cat"
```

order matrices or dataframes

Similarly, we can reorder matrices and dataframes along either (or both) dimensions.

```r
students <- data.frame(name, gender, age, height, grade, stringsAsFactors = F)
students # original data frame
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```r
cat("\nlast row first [c(4,1,2,3),]\n")
```

```
##
## last row first [c(4,1,2,3),]
```

```r
students[c(4, 1, 2, 3), ]
```

```
##      name gender age height grade
## 4 Behrooz      M  19     72     C
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
```

```r
cat("\nput grade in 2nd column [,c(1,5,2,3,4)]\n")
```

```
##
## put grade in 2nd column [,c(1,5,2,3,4)]
```

```r
students[, c(1, 5, 2, 3, 4)]
```

```
##      name grade gender age height
## 1   Leila     A      F  18     60
## 2   Vijay     D      M  23     70
## 3   Shira     B      F  48     65
## 4 Behrooz     C      M  19     72
```

And we can sort using the order or one (or more) columns.

```r
students <- data.frame(name, gender, age, height, grade, stringsAsFactors = F)
students # original data frame
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 2   Vijay      M  23     70     D
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
```

```r
cat("\ngrade order\n")
```

```
##
## grade order
```

```r
grade.ord <- order(students$grade, decreasing = T)
grade.ord
```

```
## [1] 1 3 4 2
```

```r
# sort by grade order
students[grade.ord, ]
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
## 2   Vijay      M  23     70     D
```

```r
# sort by grade order, same result
students[order(students$grade, decreasing = T), ]
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
## 2   Vijay      M  23     70     D
```

```r
# sort by gender, then age
students[order(students$gender, students$age), ]
```

```
##      name gender age height grade
## 1   Leila      F  18     60     A
## 3   Shira      F  48     65     B
## 4 Behrooz      M  19     72     C
## 2   Vijay      M  23     70     D
```